Face Presence Detection Module

Phase 3/4 — AI Interview Monitoring System

Python 3.8+	OpenCV 4.x	MediaPipe 0.10+	Flask 3.x	MIT License

Overview

The Face Presence Detection Module provides real-time monitoring of candidate visibility during AI-assisted interviews. It continuously analyzes the video feed to detect whether the candidate remains in frame, flags absences with timestamps, and exposes results via a REST API and live MJPEG stream.

Built as part of Phase 3/4 of the Interview Monitoring System, this module is designed to integrate directly with the broader proctoring pipeline.

Features

- **Real-time detection** Processes every frame with sub-100ms latency using OpenCV Haar Cascade
- **Configurable thresholds** All sensitivity parameters adjustable at runtime via the /config endpoint
- **Absence timing** Tracks duration_absent_sec with a configurable grace period to ignore blinks and brief occlusions
- **Smoothing buffer** 10-frame rolling window prevents flicker from single-frame detection misses
- **Frame preprocessing** CLAHE + unsharp mask applied before detection to handle poor lighting and camera blur
- **Dual backend** OpenCV Haar Cascade (zero setup) with automatic upgrade to MediaPipe when a .tflite model file is provided
- **REST API** Five Flask endpoints: /status, /stream, /stats, /reset, /config
- **MJPEG stream** Annotated live video feed with presence overlay, consumable by browser or OpenCV

Project Structure

```
face-detection/
├── face_presence_detector.py   # Core detection module
├── app.py                      # Flask REST API + MJPEG stream
├── test_detector.py            # Unit + integration test runner
└── requirements.txt            # Python dependencies
```

Prerequisites

Python	3.8 or higher
pip	Latest version recommended
Webcam / Video	USB webcam (index 0) or any .mp4 / .avi file
OS	Windows 10+, macOS 12+, Ubuntu 20.04+

Installation

1. Clone or download the project

```
git clone https://github.com/your-username/face-detection.git
cd face-detection
```

2. Create and activate a virtual environment

```
# Windows
python -m venv venv
venv\Scripts\activate

# macOS / Linux
python3 -m venv venv
source venv/bin/activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

Running the Project

Run unit tests first

Always run tests before starting the server to confirm the environment is set up correctly:

```
python test_detector.py               # 5 unit tests
python test_detector.py --video sample.mp4   # integration test on video file
python test_detector.py --webcam      # live webcam test (press Q to quit)
```

Start the Flask API server

```
python app.py
```

The server starts on `http://127.0.0.1:5050`. You will see:

```
 * Running on http://127.0.0.1:5050
 * Running on http://10.x.x.x:5050
Press CTRL+C to quit
```

This is expected — the server is running successfully.

API Reference

GET /status

Returns the latest presence detection result as JSON.

```
{
    "face_present": false,
    "duration_absent_sec": 4.0,
    "confidence": 0.0,
    "timestamp": "2025-01-01T10:00:00Z",
    "frame_index": 120,
    "face_bbox": null
}
```

GET /stream

MJPEG video stream with detection overlay. Open directly in a browser or consume with OpenCV. Add `?source=0` for webcam or `?source=path/to/file.mp4` for a video file.

GET /stats

Returns cumulative session statistics.

```
{
    "total_frames": 450,
    "absent_frames": 32,
    "absent_percent": 7.1,
    "backend": "haar"
}
```

POST /reset

Resets all detection state and counters. Call between interview sessions.

POST /config

Updates detection thresholds at runtime without restarting the server.

```
curl -X POST http://localhost:5050/config \
  -H "Content-Type: application/json" \
  -d '{"absent_frame_threshold": 10, "presence_ratio_threshold": 0.4}'
```

Configuration Reference

All thresholds live in DetectorConfig inside face_presence_detector.py and can also be updated at runtime via /config.

Parameter	Default	Description
absent_frame_threshold	8	Consecutive absent frames before absence is confirmed — grace period for occlusion
smoothing_window	10	Number of recent frames used to smooth the presence decision
presence_ratio_threshold	0.35	Fraction of smoothing window that must show a face to be considered present
min_face_size_ratio	0.08	Minimum face height as a fraction of frame height; filters out tiny background detections
haar_min_neighbors	4	Haar cascade minNeighbors — higher reduces false positives but may miss partial faces
enhance_frame	True	Apply CLAHE and sharpening before detection (recommended for poor lighting)
mediapipe_model_path	None	Optional path to blaze_face_short_range.tflite for MediaPipe backend

Edge Cases & How They Are Handled

Poor lighting	CLAHE histogram equalization applied to the L channel in LAB colour space before detection, followed by a mild unsharp mask to restore contrast
Camera blur	Sharpening kernel (3x3 unsharp mask) applied during preprocessing restores edge definition for blurry frames
Partial face visibility	min_face_size_ratio set to 0.08 accepts faces as small as 8% of frame height; Haar cascade detects frontal faces even when partially cropped
Temporary occlusion	absent_frame_threshold provides a configurable grace period; the 10-frame smoothing buffer prevents single-frame misses from triggering an absence event
Rapid head movement	smoothing_window averages detections across frames, making the presence signal robust to fast motion blur

VS Code Quick Start

Open the project folder in VS Code, then use the integrated terminal (Ctrl + `) for the following sequence:

1. **Open terminal** Ctrl + ` (backtick)
2. python -m venv venv
3. venv\Scripts\activate (Windows) or source venv/bin/activate (macOS/Linux)
4. pip install -r requirements.txt
5. python test_detector.py — all 5 tests should pass
6. python app.py
7. **New terminal tab** Ctrl + Shift + ` — call curl http://127.0.0.1:5050/status

Upgrading to MediaPipe Backend

The module defaults to OpenCV Haar Cascade which requires no extra files. For higher accuracy (especially on angled or partially lit faces), provide a MediaPipe model:

- Pass `mediapipe_model_path="blaze_face_short_range.tflite"` to DetectorConfig

License

MIT License — free to use, modify, and distribute with attribution.